
Adaptive Layer Execution: Learning to Order Computation in Language Models

Abdellah Rahmani¹, Sean McLeish², Ang Li³, Sanae Lotfi³,
Pavel Izmailov³, Pascal Frossard¹, Micah Goldblum⁴

¹EPFL, ²University of Maryland,
³New York University, ⁴Columbia University

Abstract

Scaling language models unlocks impressive capabilities, but existing recurrent-depth models apply a predefined number of transformer layers in a fixed order. This static design limits their ability to adapt and specialize their computation to different inputs. We present Adaptive Layer Execution (ALE), an approach for building deep recurrent models that use a single shared router, applied across recurrent depths, to decide which transformer layers to call, in what order, and how many times, conditioned on the input. By reasoning implicitly in latent space, ALE scales test-time computation while tailoring its processing to each prompt. We evaluate ALE in two regimes, using a single transformer block for both the prelude and the coda, the layers that map tokens into and out of the recurrent latent space, and a fixed recurrent depth of 22. In the first, trained on 10B tokens, ALE consistently outperforms a 24 layer vanilla model even as we shrink its pool of transformer layers from 22 down to 8: rather than adding parameters, it reuses this smaller pool, adaptively selecting and repeating layers per input. In the second, a higher capacity setting trained on 30B tokens with 46 transformer blocks, ALE further surpasses vanilla models on downstream tasks. Together, these results show that input-dependent layer selection is a promising route to language models that are at once more parameter efficient and more adaptive than fixed depth architectures.

1 Introduction

People naturally put more effort into some problems than others. We can think over long stretches of time by saying our thoughts out loud or writing them down step by step, but a lot of thinking also happens quietly inside the brain, through complex, repeating patterns of activity, before we say a single word of an answer.

Early work on making language models more capable focused on scaling Transformer networks up to hundreds of billions of parameters, which drove impressive gains in reasoning and few-shot learning [4, 5, 28, 31, 16, 8, 17]. These gains, however, come with two difficulties. The first is cost: the memory and compute these models require put training and deployment out of reach for anyone without very large data centers [32, 30]. The second is rigidity: a standard model is a stack of layers, each with its own parameters, always executed in the same order. This is inefficient, because different inputs may call for the same operations in a different order, and a fixed-order model forced to apply an operation at several different depths must duplicate the parameters for it at each one, creating redundancy [29].

The first challenge motivates *parameter efficiency* [9, 1, 39, 11, 26], which reduces or shares weights across the network. A common instance is *layer tying*, where one set of weights is reused across many layers [9, 25, 18, 1, 41], and a natural way to realize it is the *Recursive Transformer* [1, 10, 19, 44, 36, 15], which repeatedly applies a single shared block of layers. This weight sharing keeps the

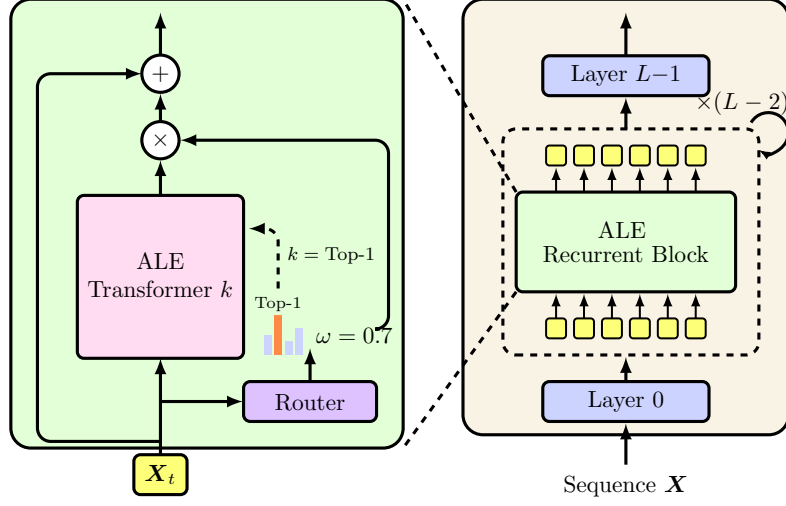


Figure 1: **Overview of Adaptive Layer Execution (ALE).** (Right) An input sequence $\mathbf{X}$ passes through Layer 0, a shared *ALE Recurrent Block* applied $L - 2$ times, and Layer $L - 1$. (Left) Zoom on one recursion step: a shared router reads $\mathbf{X}_t$ (the output of *ALE Recurrent Block* t) and, via a Top-1 selection, routes the sequence to a single transformer layer k with gating weight ω ; the layer’s output is scaled by ω and added to $\mathbf{X}_t$ through a residual connection. Selecting k per input makes the recursion depth personalized to each sequence.

parameter count low, while unrolling the block to greater depth provides a natural mechanism for scaling computation at test time.

Recursion, however, addresses only the first challenge. Existing recursive models apply the *same* block, whether for a fixed or an input-dependent number of steps, and that block runs the *same* layers in the *same* order for every prompt, leaving which layers to apply, and in what order, fixed rather than learned. The second challenge, rigidity, therefore remains open. In this paper we introduce *Adaptive Layer Execution* (ALE), a deep recurrent architecture that pairs a global dictionary of transformer layers with a compact router trained end-to-end *without explicit supervision*. At each recurrent step, the router selects which block should process the sequence next, deciding from a compact summary of it: windowed, mean-pooled hidden states.

Crucially, ALE routes per sequence rather than per token. Token-wise routing, as in Mixture-of-Experts (MoE), is ill-suited to routing whole blocks: if different tokens take different blocks, their queries and keys come from different attention projections and can no longer be compared coherently. Applying one decision to all positions avoids this, while still letting ALE spend computation adaptively [15], iterating longer on harder inputs and reusing a shared dictionary instead of storing the same operation at many depths. ALE thus buys capability through sequential reuse rather than ever-larger weights.

Conceptually, ALE interpolates between reuse and sparsity via the dictionary size $|\mathcal{B}|$ relative to the iteration count T : when $|\mathcal{B}| > T$ some blocks are unused on a given pass and ALE is FLOP-sparse, and when $|\mathcal{B}| < T$ it reuses blocks every pass, recovering the savings of depth-recurrence. Unlike feedforward [39], ALE’s router selects entire blocks with noisy top-1 routing, without activating multiple experts per step. Its closest precursor is Mixture-of-Depths [35], which routes a binary skip, execute or repeat decision per layer; ALE generalizes this to choosing which block, in what order, and how many times over a shared dictionary, a larger decision space that adds scheduling flexibility and cuts redundancy.

Our contributions are summarized as follows:

- We propose *Adaptive Layer Execution* (ALE), a deep recurrent architecture in which a learned router selects, at each recurrent step, which block from a shared dictionary of

transformer layers to apply, making *which* layers are used, *in what order*, and *how many times* conditional on the input.

- We show that this router can be trained end-to-end during pre-training, without explicit supervision or auxiliary routing labels, from only a compact windowed, mean-pooled summary of the sequence, using noisy top-1 routing to obtain a gradient signal.
- Ablations confirm that the router learns meaningful, input dependent routing: neither randomly chosen paths nor a fixed most frequently selected path matches its performance.
- We provide empirical evidence that ALE outperforms vanilla, non-recursive baselines in two regimes: at a fixed recurrent depth it keeps winning even as the pool of available layers shrinks (10B tokens), and it further surpasses vanilla models on downstream tasks in a higher-capacity setting (30B tokens).

2 Related work

Recursive Transformers. Parameter sharing offers a complementary route to efficiency [9, 25, 43, 41, 1, 23]. The Universal Transformer showed that repeatedly applying a single block can rival the expressiveness of a deep stack of distinct layers [9]. This idea reappears as the looped Transformer, whose iterated structure has proven surprisingly capable: looped models can emulate programmable computers [19], learn iterative data-fitting procedures [44], extrapolate to far longer inputs on algorithmic tasks [10], and shed light on in-context learning by approximating multi-step optimizers [14]. Since naive weight tying can degrade accuracy, [1] attach per-loop low-rank (LoRA) adapters [22] to obtain Relaxed Recursive Transformers, and recent work shows that recurrent depth is an effective substrate for latent reasoning [15]. Most of these studies treat recursion purely as a way to save parameters. A second opportunity lies in the recursion process itself: building on early-exiting [37] and compute routing [35], the amount of recursion can be adapted to each input, spending computation only where it is beneficial.

Routing mechanisms. Routers have become the standard tool for adaptive computation in LLMs, most prominently in sparse Mixture-of-Experts (MoE) layers [39, 27, 7, 49], where a learned gate sends each token to a small subset of expert networks, expanding capacity at nearly constant cost. Early MoE designs use token-choice routing, in which the gate picks the top- k experts for a token from its hidden state [27, 11, 24]; this tends to overload some experts while underusing others, requiring auxiliary load-balancing losses. Expert-choice routing reverses the assignment, each expert selects the tokens it will process, yielding exact load balancing and better utilization [48, 21]. The same machinery has been repurposed to decide which layers to skip through a trainable router [46, 35, 12]; unlike confidence-based early exit, these methods enforce a fixed compute budget by capping how many tokens each depth may process.

Our approach. These threads address our two challenges separately: recursion tackles parameter cost, routing tackles adaptive computation. ALE unifies them, resolving the rigidity that neither addresses, since existing recursive models still apply the *same* layers in the *same* order for every input. ALE differs in three ways. First, whereas MoE and depth routing methods route per token, ALE routes per *sequence* from a compact windowed, mean-pooled summary, so every position follows a coherent path over whole blocks. Second, rather than repeating a single tied block, at each step the router *dynamically* picks which block to apply from a shared dictionary, making the recursive path, which layers, in what order, and how many times, conditioned on the input. Third, this generalizes Mixture-of-Depths [35] from a per-layer skip-or-execute decision to a strictly larger space over block choice, order, and repetition. ALE thus turns a fixed recursion loop into a personalized computation that is at once parameter efficient and adaptive.

3 Adaptive Layer Execution

3.1 Preliminary

Transformer. A standard decoder-only Transformer [42] forms its token representations by passing them through a stack of L distinct layers, each pairing a causal multi-head self-attention sublayer (MHA) with a position-wise feed-forward network (FFN). Let h^l denote the hidden state of a given

token at depth l , and let ϕ^l collect the parameters of layer l . Each layer applies a residual update, so the representation evolves as

$$h^{l+1} = h^l + f^l(h^l; \phi^l), \quad f^l(h^l; \phi^l) = \text{MHA}(h^l) + \text{FFN}(h^l + \text{MHA}(h^l)), \quad (1)$$

where $f^l(\cdot; \phi^l)$ is the residual function computed by layer l . In practice each sublayer is also normalized; contemporary LLMs adopt the pre-norm placement, giving

$$a^l = h^l + \text{MHA}(\text{Norm}(h^l)), \quad h^{l+1} = a^l + \text{FFN}(\text{Norm}(a^l)), \quad (2)$$

which reduces to Eq. (??) when the normalization is folded into f^l . The input h^0 is the token embedding, and the final state h^L is mapped by a linear head and a softmax to the next-token distribution.

Parameter-sharing strategies. Recursive Transformers [1, 10, 19, 44, 36] lower the parameter count by reusing the same layers at multiple depths. Rather than learning L separate weight sets, they group the network into N_r recursion blocks that all draw from a single shared parameter pool Φ' . Unrolling these blocks raises the effective depth, and thus the amount of computation, while leaving the parameter budget unchanged. There is four schemes: Cycle and Sequence, together with their Middle-Cycle and Middle-Sequence variants. The closest scheme to our MoL method is the Middle-Cycle (Figure 2) used in [15].

3.2 Recurrent architecture

In this section we describe our proposed architecture for a Transformer with latent recurrent depth, together with its design choices and router. Throughout, we denote the sequence length by n , the model’s hidden dimension by h , and its vocabulary by the set V .

3.2.1 ALE Recurrent Block

The architecture is built from decoder-only Transformer blocks [42, 34], organized into three functional groups. The *prelude* P embeds the input tokens into a latent space and may comprise several Transformer layers (one in our experiments). The *core*, our *ALE Recurrent Block* R , is the central unit of computation: it repeatedly updates a latent state $s \in \mathbb{R}^{n \times h}$. Finally, the *coda* C maps the latent state back out of latent space, again optionally using several layers (one in our experiments), and holds the model’s prediction head. Placing the core between the prelude and coda, and pairing a dictionary of candidate Transformer blocks with a predefined maximum recurrent depth, yields a deep recursive computation field. Given a number of recurrent iterations T and an input sequence $x \in V^n$, these three groups produce the output probabilities $p \in \mathbb{R}^{n \times |V|}$ as

$$\begin{aligned} s_0 &= P(x) \\ s_i &= R(s_{i-1}) \quad \text{for } i \in \{1, \dots, T\} \\ p &= C(s_T). \end{aligned}$$

The prelude maps the input to an initial state s_0 ; the core block R is then applied T times, each pass taking the latent state s_{i-1} and returning an updated state s_i ; finally, the coda processes the last state s_T into the next-token probabilities.

Within the prelude and coda we follow standard Transformer design. Each block may contain several layers (one in our experiments), and every layer uses causal self-attention with RoPE [40] (base 50000) and a gated SiLU MLP [38], with RMSNorm [47] for normalization.

Routing inside the recurrent block. Within the ALE Recurrent Block, the latent state s_{i-1} is first passed to the router, which decides which Transformer block should process the sequence at the

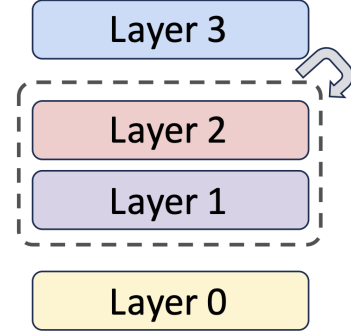


Figure 2: **Middle-Cycle recursion** [2]. The first and last layers (Layer 0, Layer 3) keep dedicated weights, while the middle layers (Layer 1–2) form a shared block (dashed box) applied cyclically, as shown by the loop arrow, raising effective depth without adding parameters.

next iteration. The router returns the top-1 index k and a scalar weight ω giving the confidence of that choice. A discrete top-1 selection is not differentiable, so the router would otherwise receive no gradient. We therefore scale the chosen block’s output by ω and add it through the residual connection, so that the router weight sits on the computation path and gradients flow back into it, following the gating mechanism used to train sparse Mixture-of-Experts models [39, 11]. To our knowledge, this is the first time such gating is used to route *entire transformer blocks per sequence within a recurrent depth*, rather than to select FFN experts per token:

$$\begin{aligned} k, \omega &= \text{Router}(\mathbf{s}_{i-1}), \quad i \in \{1, \dots, T\} \\ \mathbf{s}_i &= \mathbf{s}_{i-1} + \omega f^k(\mathbf{s}_{i-1}; \phi^k), \end{aligned} \quad (3)$$

where $f^k(\mathbf{s}_{i-1}; \phi^k)$ is the residual function of block k , defined in Eq. (1).

Routing and causality. The router forms its decision by pooling information across the sequence, so a choice computed from the full latent state depends on all positions, including future ones. During pretraining we compute one routing decision from the entire sequence and apply it at every position; this deliberately breaks causality, since a token’s computation can then depend on tokens that follow it. We accept this at training time because it lets the router see complete sequences and learn meaningful, content-dependent routing efficiently over full training examples. At inference, however, causality is preserved. The routing decision is computed once, from the prompt alone, and then held fixed throughout generation. Because the prompt is fully available before any answer token is produced, conditioning on it leaks no future information: every generated token is influenced only by the prompt and by tokens that precede it, so the autoregressive property holds exactly during answer generation.

3.2.2 ALE Router

As shown in Figure 1, the ALE Recurrent Block relies on a single router, shared across all recurrent steps, to decide which transformer block from the dictionary B processes the sequence next. Routing is performed once per sequence rather than per token, and this choice is deliberate: if tokens were routed individually, tokens of the same sequence would pass through blocks with different attention projections, so queries and keys would no longer live in a shared space and attention could not compare them coherently (a QK mismatch). A single sequence-level decision keeps every position on the same computation path. To make this decision cheaply while preserving content information, the router operates on a windowed summary of the previous state $\mathbf{s}_{i-1}$. The n token positions are split into W contiguous windows $\{\Omega_w\}_{w=1}^W$, each summarized by its mean $\mathbf{m}_w = \frac{1}{|\Omega_w|} \sum_{t \in \Omega_w} \mathbf{s}_{i-1}^t \in \mathbb{R}^h$. A linear map $\text{Router}: \mathbb{R}^h \rightarrow \mathbb{R}^{|B|}$ casts one vote per window, and votes are aggregated by averaging logits:

$$\mathbf{z} = \frac{1}{W} \sum_{w=1}^W \text{Router}(\mathbf{m}_w). \quad (4)$$

Hard top-1 selection is prone to premature convergence: a block that wins early receives all the gradient and keeps winning, so much of the dictionary is never explored. Following noisy gating [39], we therefore perturb the logits during training with learned, input-dependent Gaussian noise,

$$\tilde{\mathbf{z}} = \mathbf{z} + \nu(W_\epsilon \mathbf{z} + \mathbf{b}_\epsilon) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I), \quad (5)$$

where ν is the softplus function. The noise projection is zero-initialized, so training begins noise-free and the model learns how much exploration to inject; the noise scale is clamped for stability, and at inference the noise is disabled ($\tilde{\mathbf{z}} = \mathbf{z}$). The routing distribution and decision, computed in float32 for numerical stability, are

$$\mathbf{p} = \text{softmax}(\tilde{\mathbf{z}}), \quad k = \arg \max_{b \in \{1, \dots, |B|\}} p_b, \quad \omega = p_k, \quad (6)$$

and the selected index k and gate ω enter the recurrent update of Eq. (3). We default to $W = 8$ windows (clamped by the sequence length). Router weights use Xavier-uniform initialization [20]

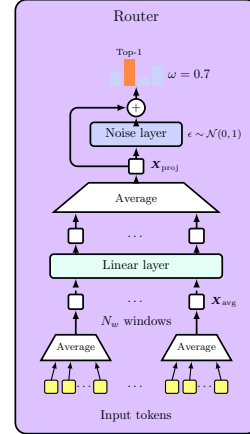


Figure 3: ALE Router architecture.

with zero biases, and the router is trained end-to-end with the rest of the model, without explicit supervision.

4 Experiments

Table 1: **ALE results at 10B (a) and 30B (b) training tokens.** Models are evaluated few-shot on PIQA (PQ), ARC-Easy (ARE), and HellaSwag (HS); higher is better ($\uparrow$). T is the maximum recurrent depth ($L - 2$ in Figure 1), $|\mathcal{B}|$ the number of transformer experts the top-1 router selects from, *Param* the total parameters, and *layers* the total transformer layers; the training-token budget N_{tok} is given in each panel header. *Random* is chance-level accuracy and *Vanilla* a non-recursive Transformer of matched depth. *Rand ALE* picks a random expert at each recursion step and *TopPath ALE* always follows the most-frequently-selected path; both isolate the learned router’s contribution at identical size and depth. ALE rows are shaded gray; bold marks the best value per column within each panel.

	Recursion		Pretrain		Few-shot Accuracy $\uparrow$		
Models	T	$ \mathcal{B} $	Param	layers	PQ	ARE	HS
Random	-	-	-	-	50.0	25.0	25.0
(a) Pre-trained on $N_{tok} = 10B$ tokens							
Vanilla	-	-	410M	24	57.8	41.8	28.3
TopPath ALE	22	22	410M	24	51.4	26.1	25.3
ALE	22	22	410M	24	61.5	46.7	30.1
	22	10	205M	12	60.9	46.4	30.1
	22	6	155M	8	59.0	43.3	28.7
(b) Pre-trained on $N_{tok} = 30B$ tokens							
Vanilla	-	-	410M	24	64.4	55.6	35.0
Rand ALE	22	46	706M	48	51.5	25.8	25.8
TopPath ALE	22	46	706M	48	53.2	27.3	25.1
ALE	22	22	410M	24	63.1	53.5	35.6
	22	46	706M	48	65.0	59.1	36.8

Setup. All models are decoder-only Transformers following the Llama design [28]: causal self-attention with RoPE (base 50,000) [40], gated SiLU MLPs [38], and RMSNorm [47]. ALE has one transformer layer in prelude and one coda, a maximum recurrent depth fixed to $T = 22$ in all experiments, and a noisy top-1 router with $W = 8$ windows selecting from a dictionary of $|\mathcal{B}|$ transformer experts. We pretrain all models from scratch on FineWeb [33] for either 10B or 30B tokens and report few-shot accuracy on PIQA (PQ) [3], ARC-Easy (ARE) [6], and HellaSwag (HS) [45], evaluated with the LM Evaluation Harness [13]. We compare against three references trained under identical settings: *Random*, the chance-level accuracy of each benchmark; *Vanilla*, a standard non-recursive Transformer of matched depth; and two routing ablations, *Rand ALE* and *TopPath ALE*, described below. Results are reported in Table 1.

ALE outperforms Vanilla with up to $2.6\times$ fewer parameters. Table 1 evaluates parameter efficiency at 10B training tokens, holding the recurrent depth fixed at $T = 22$ while shrinking the expert dictionary from $|\mathcal{B}| = 22$ to 10 to 6, which reduces the model from 410M parameters (24 layers) to 205M (12 layers) and 155M (8 layers). At matched size, ALE clearly outperforms the 410M Vanilla baseline on all three benchmarks (61.5 vs. 57.8 on PQ, 46.7 vs. 41.8 on ARE, 30.1 vs. 28.3 on HS; average 46.1% vs. 42.6%). More importantly, the advantage survives aggressive parameter reduction: the 205M model still beats Vanilla on every benchmark, and even the 155M model, with $2.6\times$ fewer parameters and one third of the unique layers, remains ahead (59.0 vs. 57.8, 43.3 vs. 41.8, 28.7 vs. 28.3). Because depth is held constant, the model compensates for the smaller dictionary by reusing its experts adaptively: capability comes from input-conditioned reuse of a small pool rather than from storing more unique weights.

Gains persist and grow with capacity. For 30B tokens. At matched size (410M), ALE is competitive with Vanilla, trailing slightly on PQ and ARE (63.1 vs. 64.4 and 53.5 vs. 55.6) while improving HS (35.6 vs. 35.0). Enlarging the dictionary to $TE = 46$ (706M parameters, 48 layers) makes ALE the strongest model on every benchmark (65.0 on PQ, 59.1 on ARE, 36.8 on HS; average 53.6% vs. 51.7% for Vanilla). This traces a consistent picture across scales: growing the expert dictionary is an effective capacity axis for ALE, while shrinking it degrades accuracy only gracefully.

Ablations: the learned router is what drives the gains. A natural concern is that ALE’s improvements stem from its recursive depth or parameter count rather than from routing itself. To isolate the router’s contribution, Table 1 compares ALE against two ablations at identical size, depth, and training budget (706M, 48 layers, $RD = 22$, 30B tokens): *Rand ALE*, which replaces the learned decision with a uniformly random expert at every recursion step, and *TopPath ALE*, which discards input-dependence and always follows the single most-frequently-selected path. Both collapse toward chance level (averages of 34.4% and 35.2%, against 25–50% chance and 53.6% for ALE), losing more than 30 points on ARC-Easy alone; the same failure appears at 10B tokens, where TopPath ALE reaches only 51.4/26.1/25.3. Two conclusions follow. First, capacity and depth alone explain none of the gain: the ablations have exactly as much of both. Second, no single static path through the dictionary suffices, since even the empirically most popular path fails; the router’s decisions are therefore both meaningful and genuinely input-dependent, adapting the composition and order of experts to each sequence.

5 Conclusion

We introduced Adaptive Layer Execution (ALE), a deep recurrent architecture in which a shared, noisy top-1 router selects, at each recurrent step, which transformer block from a common dictionary should process the sequence, making which layers are applied, in what order, and how many times conditional on the input. Routing at the sequence level keeps every position on a coherent computation path and avoids the query–key mismatch of per-token routing, while the router itself is trained end-to-end without any explicit supervision. Empirically, ALE addresses both challenges that motivated it. At a fixed recurrent depth, it outperforms a vanilla Transformer of matched depth even as the expert dictionary shrinks by a factor of $2.6\times$ in parameters, and with a larger dictionary trained on 30B tokens it surpasses the vanilla baseline on all three few-shot benchmarks. Our ablations show that neither random routing nor the single most popular path recovers this performance, confirming that the learned, input-dependent routing is the source of the gains rather than depth or capacity alone.

The main limitation of our current design lies in pre-training: the router forms its decision from a summary of the full sequence, so a token’s computation can depend on positions that follow it, breaking causality during training even though inference, which routes from the prompt alone, remains fully autoregressive. Our future work will target this train-time causality gap directly, for instance by restricting the router to causal (prefix- or window-limited) summaries or by segment-wise routing schemes that preserve the autoregressive property throughout pre-training. Beyond this, natural extensions include scaling ALE to larger models and token budgets, richer routing decisions such as early exit and per-step recursion counts, and a deeper analysis of the paths the router discovers. Together, these directions move toward language models that allocate their computation the way the problem demands: reusing a small dictionary of layers, adaptively, per input.

References

- [1] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA, 2024.
- [2] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoung Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, et al. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *Advances in Neural Information Processing Systems*, 38:96572–96617, 2026.
- [3] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [4] Tom B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] Aakanksha Chowdhery et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24, 2023.
- [6] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge, 2018.
- [7] Damai Dai, Li Dong, Shuming Ma, Bo Zheng, Zhifang Sui, Baobao Chang, and Furu Wei. StableMoE: Stable routing strategy for mixture of experts. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7085–7095, 2022.
- [8] DeepSeek-AI. DeepSeek-V3 technical report, 2024.
- [9] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *International Conference on Learning Representations (ICLR)*, 2019.
- [10] Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization, 2024.
- [11] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 2022.
- [12] Gadhikar et al. Verify: Gadhikar et al. (2024), router-based layer skipping – exact reference not confirmed, 2024. Placeholder – replace with the correct entry from the MoR bibliography (arXiv:2507.10524).
- [13] Leo Gao, Jonathan Tow, Baber Abbasi, et al. The language model evaluation harness. <https://github.com/EleutherAI/lm-evaluation-harness>, 2024.
- [14] Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. Can looped transformers learn to implement multi-step gradient descent for in-context learning? In *International Conference on Machine Learning (ICML)*, 2024.
- [15] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025.
- [16] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.
- [17] Gemini Team, Google. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.
- [18] Gholami and Omar. Verify: Gholami and omar (2023) – exact title/authors/venue not confirmed, 2023. Placeholder – replace with the correct entry (see note below).

- [19] Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*, 2023.
- [20] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.
- [21] Guo et al. Verify: Guo et al. (2025), expert-choice routing – exact reference not confirmed, 2025. Placeholder – replace with the correct entry from the MoR bibliography (arXiv:2507.10524).
- [22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [23] Andrew Jaegle, Felix Gimeno, Andy Brock, Andrew Zisserman, Oriol Vinyals, and João Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, 2021.
- [24] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, et al. Mixtral of experts, 2024.
- [25] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*, 2020.
- [26] Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.
- [27] Dmitry Lepikhin, HyukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding, 2020.
- [28] Llama Team, AI @ Meta. The Llama 3 herd of models, 2024.
- [29] Xin Men, Mingyu Xu, Qingyu Zhang, Qianhao Yuan, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20192–20204, 2025.
- [30] Ali Momeni et al. Training of physical neural networks, 2024.
- [31] OpenAI. GPT-4 technical report, 2023.
- [32] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training, 2021.
- [33] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale, 2024.
- [34] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.
- [35] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.
- [36] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations (ICLR)*, 2025.

- [37] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [38] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [39] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*, 2017.
- [40] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [41] Sho Takase and Shun Kiyono. Lessons on parameter sharing across layers in transformers, 2021.
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [43] Yingce Xia, Tianyu He, Xu Tan, Fei Tian, Di He, and Tao Qin. Tied transformers: Neural machine translation with shared encoder and decoder. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 33, pages 5466–5473, 2019.
- [44] Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms, 2023.
- [45] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- [46] Dewen Zeng, Nan Du, Tao Wang, Yuanzhong Xu, Tao Lei, Zhifeng Chen, and Claire Cui. Learning to skip for language modeling, 2023.
- [47] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.
- [48] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [49] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models, 2022.